

container (in our case, I assume it is Tomcat). Rename it to *solr.war* and deploy the file.

Next, you need to set up the home folder which contains all the configuration files, etc. To do this, you can either go to *apache-solr-1.4.1/example*, copy the *solr* directory, and paste it into the *bin* folder of Tomcat—or you can set the Java system property of the *solr.solr.home* parameter to point to the location of your home folder.

Now, visit <http://localhost:8080/solr/admin>. You should see the Solr Admin console (see Figure 2), and you're done. If you're still not able to get this working, you can try an example set-up using the lightweight Jetty server, which can be run using the following command in the example folder:

```
java -jar start.jar
```

This will set up Jetty to listen on port 8983 instead of 8080, so you will have to replace the port number in the corresponding examples. You may explore the Admin console now, which consists of several tabs and a query box. At this point, queries will not return results, since we haven't yet indexed anything; we will do so later.

Architecture

Solr consumes data in the form of documents. A document is like a basic unit of information that is fed into Solr. It consists of fields, which are, in turn, a more specific piece of information.

The process shown so far, is a simplified version of the installation procedure, and probably not how you would like it in a production system. For such purposes, you should refer the actual documentation on the Solr wiki.

Let's now look at some of the important files, and the directory structure:

Directory	Function
<i>solr/bin</i>	This is an optional directory for keeping replication scripts, in case we want to implement some amount of replication on our servers.
<i>solr/data</i>	This is the default location where the index of your search application will be stored. Its location can be changed in the <i>solrconfig.xml</i> file.
<i>solr/conf</i>	This is the most important directory, where all the configuration files are stored.

The two most important files in the *conf* directory are as follows; each of these is extensively commented, with every parameter explained in detail:

- *solrconfig.xml*: This file handles the entire primary configuration. It is used to define request handlers,

response writers (explained below), auto-warming parameters and plug-ins.

- *schema.xml*: This file, as is obvious, dictates the 'schema' or the structure of your documents. It also specifies how each field in the document should be indexed, and what fields each document can contain. The file is divided into sections to define types, fields and other definitions pertaining to documents. It also specifies *Analyzers* in order to determine the manner in which each field is indexed and retrieved. Schema design is one of the most important steps while designing an application on top of Solr.

RequestHandler

A *RequestHandler* dictates what is to be done when a particular request is made. It basically processes all the interactions with the search server, including querying, indexing, etc. Examples of some of the many *RequestHandlers* available (besides the *StandardRequestHandler*) are *DisMaxRequestHandler*, *DataImportHandler*, *MoreLikeThisHandler*, *SpellCheckerRequestHandler*, etc. Each of these are built to perform specific functions; Solr even allows you to write custom handlers for your own purpose.

ResponseWriter

ResponseWriter generates and formats the response to a query, or any request made to the server. Multiple *ResponseWriters* are available to produce the data to be consumed in the form of JSON, PHP, Python, Ruby, CSV, XML and even XSLT. A custom *ResponseWriter* can also be written in a manner similar to a *RequestHandler*, by implementing an interface specified for each, in Java.

Search components

Search components are something that provide a specific functionality that can be reused with any type of handler to provide a common logic. These are the most basic extension methodologies in Solr. Each component is configured in the *solrconfig.xml* file as an addition to the *RequestHandlers*.

Indexing techniques

Indexing is feeding data into Solr's index. Proceeding with the example, let's first send the data to our installed application for indexing, with the help of the included simple *post* application. Go to the *apache-solr-1.4/example/exampledocs* folder, from the location where you initially unzipped the Solr archive. In this directory, give the following command:

```
java -Durl=http://localhost:8080/solr/update -jar post.jar *.xml
```

After the process is completed, you will be able to search for your data from the admin console. The data that gets posted to the Solr index is all the XML files that are contained